

After calling and importing all necessary packages and libraries, the system is now ready to reference decision-making algorithms for the objects.

Setting execution

```
• execution_path = os.getcwd()
• detector = ObjectDetection()
• detector.setModelTypeAsRetinaNet()
• detector.setModelPath( os.path.join(execution_path ,
"resnet50_coco_best_v2.0.1.h5"))
• detector.loadModel()
• custom_objects = detector.CustomObjects(person=True,
car=False)
• detections = detector.detectCustomObjectsFromImage(i
nput_image=os.path.join(execution_path , "image.png"),
output_image_path=os.path.join(execution_path , "image_
new.png"), custom_objects=custom_objects, minimum_per-
centage_probability=65)
```

The code shows how the execution paths are set and how the system calls the detector algorithm to study and learn the images. This is followed by codes that reference the same function for object detection and writing the code for it with a Boolean value for what should be accepted and what shouldn't be.

As you can see, the following code only looks to identify either cars or people in images. This is again possible if the system has been referenced with code for labelling and creating classification based names for all unique objects that appear in images. As a result, this can be expanded to include other sorts of objects as long as the system is trained to look for things that do not match the initially defined items in the input.

Multi-image identification with numerous objects will thus yield more complex codes, but for now, we can run with this alone.

Optimizing with Intel

You can optimise TensorFlow for Intel® hardware by changing the parallelism threads and OpenMP settings:-

```
• import tensorflow as tf
• config = tf.ConfigProto(intra_op_parallelism_
threads=NUM_PARALLEL_EXEC_UNITS, inter_op_parallel-
```